CODING PATTERNS

@aman_views

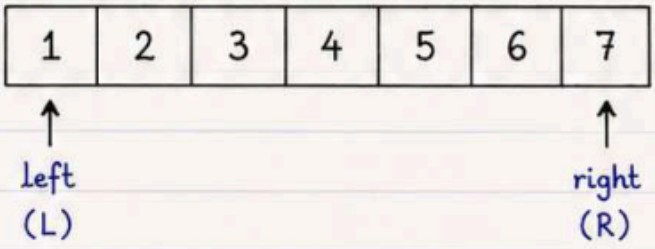
1. TWO POINTERS PATTERN

OVERVIEW

- Two Pointer technique uses two indices to traverse a data structure.
- It is used to solve problems on sorted arrays, linked lists or sequences.
- It helps to reduce time complexity from $O(n^2)$ to $O(n)$ in many cases.

VISUAL DIAGRAM

Example Array :



WHEN TO USE ?

- When the input is sorted.
- When we need pairs or triplets.
- When we need to compare elements from both ends or within a range.
- When we need to remove duplicates in-place.

JAVA TEMPLATE

```
int left = 0;
int right = n - 1;

while (left < right) {
    // your logic here

    if (condition) {
        left ++;
    } else {
        right --;
    }
}
```

RECOGNITION KEYWORDS

- Pair / Triplet
- Sorted Array
- Left / Right / Start / End
- Move towards center
- Remove duplicates
- Minimize / Maximize something

TIME COMPLEXITY

Best Case	:	O(n)
Worst Case	:	O(n)
Space	:	O(1)

FAMOUS PROBLEMS

1. Two Sum II - Input Array is Sorted
2. Container With Most Water
3. Valid Palindrome
4. Remove Duplicates from Sorted Array
5. 33um
6. Trapping Rain Water

COMMON MISTAKES

- Forgetting to move pointers (left++ or right--).
- Not handling edge cases (empty array, single element).
- Using this pattern on unsorted data.
- Infinite loop due to wrong pointer movement.

CODING PATTERNS

@aman_views

2. SLIDING WINDOW PATTERN

OVERVIEW

- Sliding Window is used to solve problems on arrays or strings where we need to consider a subarray / substring.
- We maintain a window [L, R] and slide it based on the condition.
- It helps to optimize brute force solutions from $O(n^2)$ to $O(n)$.

WINDOW TYPES

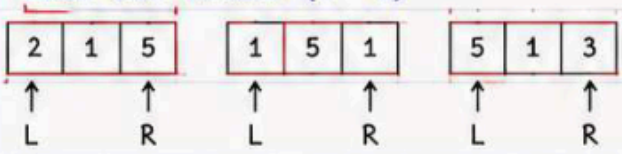
1. Fixed Size Window
   - Window size is fixed, slide one step at a time.
2. Variable Size Window
   - Window size changes based on the given condition.

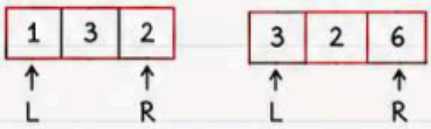
VISUAL DIAGRAM

Example Array :



Fixed Size Window (k = 3)





Variable Size Window



JAVA TEMPLATE (VARIABLE SIZE)

```
int left = 0;
int sum = 0;
for (int right = 0; right < n; right++) {
    // Add current element
    sum += arr[right];
    // Shrink window if condition violates
    while (left <= right && condition) {
        sum -= arr[left];
        left ++ ;
    }
    // Process the window
    // Update answer
}
```

RECOGNITION KEYWORDS

- Subarray / Substring
- Continuous / Contiguous
- Maximum / Minimum of window
- Longest / Shortest
- At most / At least / Exactly K
- No. of subarrays / substrings with ...

TIME COMPLEXITY

Best Case	:	O(n)
Worst Case	:	O(n)
Space	:	O(1)

Each element is visited at most twice.

FAMOUS PROBLEMS

1. Maximum Sum Subarray of Size K
2. Longest Substring Without Repeating Characters
3. Minimum Window Substring
4. Fruits Into Baskets
5. Count Number of Subarrays With Sum K

COMMON MISTAKES

- Forgetting to move left pointer.
- Not shrinking window when condition breaks.
- Off by one error in window size.
- Wrong update of answer.
- Not handling edge cases (empty, single element).

CODING PATTERNS

@aman_views

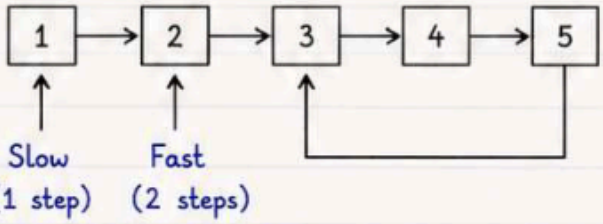
3. FAST & SLOW POINTER PATTERN

OVERVIEW

- Fast & Slow Pointer (Floyd's Cycle Detection Technique) uses two pointers moving at different speeds.
- Slow pointer moves 1 step, fast pointer moves 2 steps.
- Mainly used for linked lists or arrays to detect cycle, find middle, etc.
- Time Complexity : O(n) Space : O(1)

VISUAL DIAGRAM (LINKED LIST)

Detect Cycle in Linked List



If cycle exists, fast and slow pointers will meet at some node.

WHEN TO USE ?

- When we need to detect cycle in linked list.
- When we need to find middle element.
- When we need to find start of cycle.
- When we need to solve problems related to repeating sequences.

JAVA TEMPLATE (CYCLE DETECTION)

```
ListNode slow = head;
ListNode fast = head;
while (fast != null && fast.next != null) {
    slow = slow.next;        // move 1 step
    fast = fast.next.next;   // move 2 steps
    if (slow == fast) {
        return true;   // cycle detected
    }
}
return false;  // no cycle
```

RECOGNITION KEYWORDS

- Cycle / Loop
- Circular / Round
- Fast / Slow
- Repeating
- Middle Element
- Find Start of Cycle

TIME COMPLEXITY

Best Case	:	O(1)
Worst Case	:	O(n)
Space	:	O(1)

We traverse the list at most once.

FAMOUS PROBLEMS

1. Linked List Cycle
2. Middle of Linked List
3. Linked List Cycle II (Find Start of Cycle)
4. Happy Number
5. Find the Duplicate Number (Floyd's Algo)
6. Circular Array Loop

COMMON MISTAKES

- Forgetting to check (fast != null && fast.next != null).
- Moving fast pointer by 1 step instead of 2.
- Comparing values instead of nodes in linked list.
- Infinite loop if conditions are wrong.
- Not handling edge cases like empty list or single node.

CODING PATTERNS

@aman_views

4. BINARY SEARCH PATTERN

OVERVIEW

- Binary Search is used to search in a sorted array or search space by repeatedly dividing it in half.
- It reduces the search space by half at every step.
- Time Complexity : O(log n)
- Space Complexity : O(1) (iterative)

VISUAL DIAGRAM (SORTED ARRAY)

Search key = 23



If arr[mid] < key → search right half
If arr[mid] > key → search left half
If arr[mid] == key → element found

WHEN TO USE ?

- When the array or search space is sorted.
- When we need to find an element efficiently.
- When dealing with very large datasets.
- When the search space can be reduced based on a condition.

IMPORTANT NOTES

- Always use this formula to avoid overflow in mid calculation.

```
mid = low + (high - low) / 2
```

- Works only on sorted data or monotonic search space.
- In problems, the search space can be array indices or answer range.

JAVA TEMPLATE (ITERATIVE)

```
int low = 0;
int high = n - 1;
while (low <= high) {
    int mid = low + (high - low) / 2;  // safe mid
    if (arr[mid] == key) {
        return mid;  // found
    } else if (arr[mid] < key) {
        low = mid + 1;  // search right half
    } else {
        high = mid - 1;  // search left half
    }
}
return -1;  // not found
```

RECOGNITION KEYWORDS

- Sorted Array
- Find element / Position
- Minimum / Maximum satisfying condition
- First / Last occurrence
- Search in range
- Reduce the search space

TIME COMPLEXITY

Best Case	:	O(1)
Worst Case	:	O(log n)
Space	:	O(1) (iterative)

FAMOUS PROBLEMS

1. Binary Search (Search Insert Position)
2. Find First and Last Position of Element
3. Search in Rotated Sorted Array
4. Find Minimum in Rotated Sorted Array
5. Peak Element
6. Find Koko Eating Bananas (Answer in Range)

COMMON MISTAKES

- Wrong mid calculation: mid = (low + high) / 2 (May cause overflow)
- Using while (low < high) instead of (low <= high) in standard binary search.
- Not updating pointers correctly (low = mid, high = mid).
- Applying binary search on unsorted data.

CODING PATTERNS

@aman_views

5. BFS & DFS PATTERN

OVERVIEW

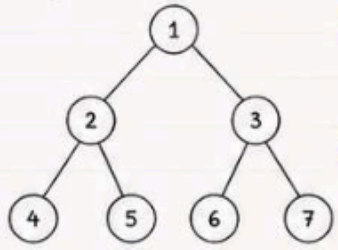
- BFS (Breadth First Search) explores level by level using Queue.
- DFS (Depth First Search) explores as deep as possible using Stack (or recursion).
- Used for trees, graphs, connected components, shortest path, etc.
- Time Complexity : O(V + E)
- Space Complexity : O(V)

BFS vs DFS (QUICK COMPARISON)

Feature	BFS	DFS
Data Structure	Queue	Stack / Recursion
Traversal	Level by Level	Go Deep First
Shortest Path	Yes (Unweighted)	No
Memory Usage	Higher	Lower
Use Cases	Shortest path, Level order, Nearest node	Cycle detection, Topological sort, Connected comp.

TREE TRAVERSAL EXAMPLE

Sample Tree :



BFS (Level Order) :

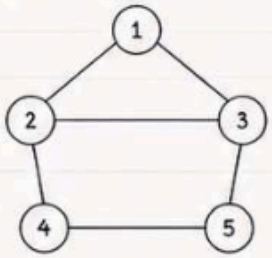
1 2 3 4 5 6 7

DFS (Preorder) :

1 2 4 5 3 6 7

GRAPH EXAMPLE

Sample Graph (Undirected) :



BFS from 1 :

1 2 3 4 6 5

DFS from 1 :

1 2 4 5 3

(Order may vary)

BFS TEMPLATE (QUEUE)

```
Queue<Node> q = new LinkedList<>();
boolean[] visited = new boolean[n];
 q.offer(start);
 visited[start] = true;
while (!q.isEmpty()) {
   Node node = q.poll();     // process node
   for (Node nei : node.neighbors) {
      if (!visited[nei]) {
          visited[nei] = true;
          q.offer(nei);
      }
   }
}
```

DFS TEMPLATE (RECURSION)

```
boolean[] visited = new boolean[n];
dfs(start, visited);

void dfs(Node node, boolean[] visited) {
   visited[node] = true;     // process node
   for (Node nei : node.neighbors) {
      if (!visited[nei]) {
          dfs(nei, visited);
      }
   }
}
```

RECOGNITION KEYWORDS

- Tree / Graph
- Connected Components
- Shortest Path (Unweighted)
- Number of Islands
- Cycle Detection
- Level Order Traversal
- Flood Fill

WHEN TO USE ?

Use BFS when :

- You need shortest path.
- You need to visit level by level.
- You need nearest node.

Use DFS when :

- You need to explore as deep as possible.
- You need cycle detection.
- You need topological ordering.

TIME COMPLEXITY

BFS	:	O(V + E)
DFS	:	O(V + E)
Space	:	O(V)

Where V = number of vertices
E = number of edges

FAMOUS PROBLEMS

1. Number of Islands
2. Clone Graph
3. Rotting Oranges
4. Word Ladder (Shortest Path)
5. Course Schedule (Cycle Detection)
6. Pacific Atlantic Water Flow

COMMON MISTAKES

- Forgetting to mark node as visited.
- Using DFS when BFS is required (shortest path problems).
- Not handling disconnected components.
- Stack overflow in DFS due to deep recursion (use iterative).
- Wrong queue operations in BFS.

CODING PATTERNS

@aman_views

6. BACKTRACKING PATTERN (BASICS)

OVERVIEW

- Backtracking is an algorithmic technique for solving problems by trying out all possible solutions.
- It builds solution step by step and backtracks (undoes) when a path does not lead to a valid solution.
- It is mainly used in problems involving combinations, permutations, subsets, partitioning, etc.

Time Complexity : Exponential (depends)

Space Complexity : O(n) (recursion stack)

BACKTRACKING FLOW

1. Choose

   Make a choice and move forward.

   ↓

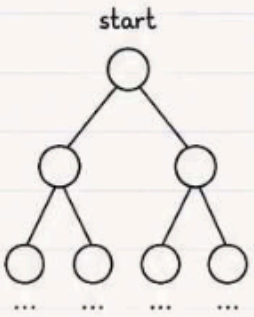
2. Explore

   Explore further with the choice.

   ↓

3. Undo (Backtrack)

   If the path is not valid, undo the choice and try other options.



WHEN TO USE ?

- When we need to explore all possible solutions.
- When the problem asks for all subsets, permutations, combinations, or arrangements.
- When we can make a choice and later undo that choice.
- When brute force with pruning can solve the problem.

KEY POINTS

- Make a choice.
- Move forward (explore).
- If not valid, undo the choice.
- Try other choices.
- Base condition helps to find a solution or stop further exploration.

GENERIC JAVA TEMPLATE

```
List<List<Integer>> result = new ArrayList<>();
List<Integer> path = new ArrayList<>();

void backtrack(int start) {
    if (/* base condition */) {
        result.add(new ArrayList<>(path));  // store
        return;
    }
    for (int i = start; i < n; i++) {
        // Choose
        path.add(i);

        // Explore
        backtrack(i + 1);

        // Undo (Backtrack)
        path.remove(path.size() - 1);
    }
}
```

RECOGNITION KEYWORDS

- All possible / every possible
- Generate / find all
- Combinations / permutations / subsets
- Arrange / partition / equal / pairs
- N-Queens / Sudoku / Word Search
- Subset sum / Combination sum

TIME COMPLEXITY

Best Case	:	O(1)
Average Case	:	Exponential
Worst Case	:	$O(2^n)$ or $O(n!)$
Space	:	O(n)

Depends on choices and depth (n = size of input)

FAMOUS PROBLEMS (BASICS)

1. Print All Subsets of a Set
2. Print All Permutations of a String
3. Count All Subsets with Given Sum
4. Combination Sum I
5. Combination Sum II

COMMON MISTAKES

- Forgetting to undo the choice (backtrack).
- Not having proper base condition.
- Modifying the same list without cloning (while storing).
- Starting loop from wrong index.
- Exploring unnecessary paths (no pruning).

CODING PATTERNS

@aman_views

7. DYNAMIC PROGRAMMING (BASICS)

OVERVIEW

- Dynamic Programming (DP) is used to solve problems by breaking them into overlapping subproblems.
- It stores results of subproblems and reuses them.
- Two main approaches :
  1. Memoization (Top-Down)
  2. Tabulation (Bottom-Up)
- Time Complexity : Usually reduces from Exponential to Polynomial.
- Space Complexity : Depends on state and storage (usually $O(n)$ or $O(n^2)$).

DP CONCEPT

1. Overlapping Subproblems
   Same subproblems are solved again and again.
2. Optimal Substructure
   Optimal solution can be constructed from optimal solutions of subproblems.
3. States
   A state represents a subproblem.
4. Transition
   Relation to derive current state from previous states.
5. Base Case
   Smallest state with known answer.

DP FLOW

Understand the problem
↓
Define the state
↓
Find the recurrence relation (transition)
↓
Identify base cases
↓
Choose approach (Memoization / Tabulation)
↓
Compute the answer

GENERIC JAVA TEMPLATE (MEMOIZATION)

```
// memoization (top-down)
int solve(int idx, int[] dp, ...){
   if (base condition) return baseAnswer;
   if (dp[idx] != -1) return dp[idx];  // already solved
   // compute answer using recurrence relation
   int ans = ... ;
   dp[idx] = ans;                 // store result
   return ans;
}
// call
Arrays.fill(dp, -1);
return solve(start, dp, ...);
```

GENERIC JAVA TEMPLATE (TABULATION)

```
// tabulation (bottom-up)
int[] dp = new int[n + 1];
// base case initialization
dp[0] = baseValue;
for (int i = 1; i <= n; i++) {
   // transition
   dp[i] = ... ;
}
return dp[n];
```

RECOGNITION KEYWORDS

- Maximum / Minimum
- Count ways
- Longest / Shortest
- Number of ways to reach to the end
- Subsequence / Substring
- Partition / Knapsack
- Game problems
- "Find the optimal ..."

TIME COMPLEXITY

Top-Down (Memo) :	O(N)
Bottom-Up (Tab) :	O(N)
Space (Memo) :	O(N)
Space (Tab) :	O(N)

N = number of states

EXAMPLE 1 : FIBONACCI NUMBER

Find nth Fibonacci number.
Recurrence : fib(n) = fib(n-1) + fib(n-2)
Base Cases : fib(0) = 0, fib(1) = 1
Tabulation :

n	0	1	2	3	4	5	6
fib	0	1	1	2	3	5	8

Answer = fib[n]

EXAMPLE 2 : CLIMBING STAIRS

You can climb 1 or 2 steps.
In how many ways can you reach top of n stairs?
Recurrence : ways(n) = ways(n-1) + ways(n-2)
Base Cases : ways(0) = 1, ways(1) = 1
Tabulation :

n	0	1	2	3	4	5
ways	1	1	2	3	5	8

Answer = ways[n]

COMMON MISTAKES

- Not identifying correct state.
- Wrong recurrence relation.
- Forgetting base cases.
- Recomputing subproblems (not using DP).

TIPS

- Always start with small cases and build intuition.
- Draw recursive tree before jumping to code.
- Memoization is easier to write, Tabulation is more efficient.
- Optimize space after getting correct solution.

CODING PATTERNS

@aman_views

8. DYNAMIC PROGRAMMING (MORE)

DP OVERVIEW

- DP = Divide problem into overlapping subproblems and solve each only once.
- Three main approaches :
  1. Memoization (Top-Down)
  2. Tabulation (Bottom-Up)
  3. Space Optimization
- Choose approach based on state definition and problem constraints.

IDENTIFY DP PROBLEM

- Can the problem be broken into subproblems?
- Are subproblems overlapping?
- Can we build the answer from smaller answers?
- If YES to all → it is DP problem.

MEMOIZATION vs TABULATION

Memoization (Top-Down)	Tabulation (Bottom-Up)
• Use recursion + cache	• Use loops
• Easy to write	• No recursion stack
• Extra space for recursion stack	• Usually faster

COMMON DP STATES

- Index based → dp[i] or dp[i][j]
- Capacity based → dp[i][w]
- State + Choice → dp[i][j]
- Bitmask → dp[mask]
- Range based → dp[i][j]

EXAMPLE 1 : 0/1 KNAPSACK (TABULATION)

Given n items, weight[i], value[i], capacity W.
Find max value.

State : dp[i][w] = max value using first i items with capacity w.

Transition :

```
dp[i][w] = max( dp[i-1][w],
                value[i-1] + dp[i-1][w - weight[i-1]] )
                              if weight[i-1] <= w
```

Base : dp[0][w] = 0 , dp[i][0] = 0

Answer : dp[n][W]

EXAMPLE 2 : LCS (LONGEST COMMON SUBSEQUENCE)

Given two strings s1 and s2, find length of LCS.

State : dp[i][j] = LCS length of s1[0...i-1] and s2[0...j-1]

Transition :

```
if s1[i-1] == s2[j-1]
    dp[i][j] = 1 + dp[i-1][j-1]
else
    dp[i][j] = max( dp[i-1][j], dp[i][j-1] )
```

Base : dp[i][0] = 0 , dp[0][j] = 0

Answer : dp[n][m]

EXAMPLE 3 : COIN CHANGE (MIN COINS)

Given coins[] and amount, return min coins to make the amount.

State : dp[a] = min coins to make amount a

Transition :

dp[a] = min(dp[a - coin] + 1) for all coin <= a

Base : dp[0] = 0

Answer : dp[amount]

If not possible → dp[amount] = INF → return -1

SPACE OPTIMIZATION

- If dp[i] depends only on dp[i-1], use 1D array.
- If dp[i][j] depends only on previous row, use 2 rows (rolling array).
- Reduces space from O(n) or O(n*m) to O(1) or O(m).

Example (Fibonacci) :

dp[i] = dp[i-1] + dp[i-2]

→ use two variables : prev2, prev1

DP TIPS

- ✓ Define state clearly.
- ✓ Write down base cases first.
- ✓ Figure out the transition relation.
- ✓ Decide order of computation (bottom-up).
- ✓ Try small examples by hand.
- ✓ Optimize space if possible.

COMMON MISTAKES

- Wrong state definition.
- Missing base cases.
- Overlapping not handled (recomputing states).
- Wrong transition / missing choices.
- Array bounds / off-by-one errors.
- Using recursion for large constraints (stack overflow).

Remember : Break → Solve → Store → Reuse

CODING PATTERNS

@aman_views

9. GREEDY ALGORITHM PATTERN

OVERVIEW

- Greedy makes the best local choice at each step hoping to get global optimum.
- Works when problem has:
  - Optimal Substructure
  - Greedy Choice Property
- Faster and simpler than DP.

WHEN TO USE ?

- When greedy choice property holds.
- When local optimum leads to global optimum.
- When you can sort / prioritize choices.
- When asked for min / max / best.

EXAMPLE PROBLEM

Activity Selection Problem

Select max non-overlapping activities.

Greedy Choice : Pick activity that finishes earliest.

Example :

Activity	A1	A2	A3	A4	A5	A6
Start	1	3	0	5	8	5
End	2	4	6	7	9	9

Answer : A1, A2, A4, A5 (Max = 4)

TYPICAL GREEDY APPROACH

1. Understand the problem.
2. Identify choices.
3. Sort / prioritize choices if needed.
4. Pick the best choice.
5. Remove / update and repeat.
6. Return the result.

CODE EXAMPLE (JAVA)

Activity Selection - Select max activities

```java
class Solution {
    static class Activity implements Comparable<Activity> {
        int start, end;
        Activity(int s, int e) { start = s; end = e; }
        public int compareTo(Activity other) {
            return this.end - other.end;  // sort by end time
        }
    }

    public int maxActivities(int[][] acts) {
        int n = acts.length;
        Activity[] arr = new Activity[n];
        for (int i = 0; i < n; i++)
            arr[i] = new Activity(acts[i][0], acts[i][1]);

        Arrays.sort(arr);    // sort by end time

        int count = 1;       // take first activity
        int lastEnd = arr[0].end;

        for (int i = 1; i < n; i++) {
            if (arr[i].start >= lastEnd) {
                count++;
                lastEnd = arr[i].end;
            }
        }
        return count;
    }
}
// Time Complexity : O(n log n)
// Space Complexity : O(n)
```

KEY POINT : Make the best choice now, trust it later!

CODING PATTERNS

@aman_views

10. MONOTONIC STACK & QUEUE

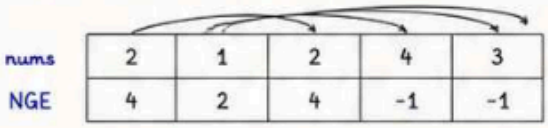
MONOTONIC STACK

Used to find Next Greater / Next Smaller element efficiently.

Example 1 : Next Greater Element (Right)

Input: nums = [2, 1, 2, 4, 3]

Output: [4, 2, 4, -1, -1]



nums	2	1	2	4	3
NGE	4	2	4	-1	-1

```
// Next Greater Element (Right)
class Solution {
  public int[] nextGreaterElements(int[] nums) {
    int n = nums.length;
    int[] res = new int[n];
    Stack<Integer> st = new Stack<>();

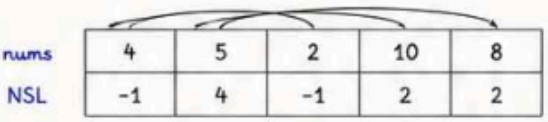
    for (int i = n - 1; i >= 0; i--) {
      while (!st.isEmpty() && st.peek() <= nums[i])
            st.pop();
      res[i] = st.isEmpty() ? -1 : st.peek();
      st.push(nums[i]);
    }

    return res;
  }
}
```

Example 2 : Next Smaller Element (Left)

Input: nums = [4, 5, 2, 10, 8]

Output: [-1, 4, -1, 2, 2]



nums	4	5	2	10	8
NSL	-1	4	-1	2	2

```
// Next Smaller Element (Left)
class Solution {
  public int[] nextSmallerLeft(int[] nums) {
    int n = nums.length;
    int[] res = new int[n];
    Stack<Integer> st = new Stack<>();

    for (int i = 0; i < n; i++) {
      while (!st.isEmpty() && st.peek() >= nums[i])
          st.pop();

      res[i] = st.isEmpty() ? -1 : st.peek();
      st.push(nums[i]);
    }

    return res;
  }
}
```

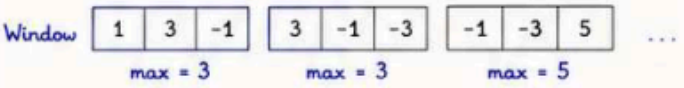
MONOTONIC QUEUE (DEQUE)

Used for problems on Sliding Window Maximum / Minimum.

Example 1 : Sliding Window Maximum

Input: nums = [1, 3, -1, -3, 5, 3, 6, 7], k = 3

Output: [3, 3, 5, 5, 6, 7]




```
// Sliding Window Maximum
import java.util.*;
class Solution {
  public int[] maxSlidingWindow(int[] nums, int k) {
    int n = nums.length;
    if (n == 0) return new int[0];
    int[] res = new int[n - k + 1];
    Deque<Integer> dq = new LinkedList<>(); // store index

    for (int i = 0, i < n; i++) {
      // remove indices out of window
      if (!dq.isEmpty() && dq.peekFirst() <= i - k)
          dq.pollFirst();

      // remove smaller elements from back
      while (!dq.isEmpty() && nums[dq.peekLast()] <= nums[i])
          dq.pollLast();
      dq.offerLast(i);

      // record max
      if (i >= k - 1)
          res[i - k + 1] = nums[dq.peekFirst()];
    }
    return res;
  }
}
```

Example 2 : Sliding Window Minimum

Input: nums = [4, 2, 12, 3, 5, 1], k = 3

Output: [2, 2, 3, 3]




```
// Sliding Window Minimum
import java.util.*;
class Solution {
  public int[] minSlidingWindow(int[] nums, int k) {
    int n = nums.length;
    if (n == 0) return new int[0];
    int[] res = new int[n - k + 1];
    Deque<Integer> dq = new LinkedList<>(); // store index

    for (int i = 0, i < n; i++) {
      if (!dq.isEmpty() && dq.peekFirst() <= i - k)
          dq.pollFirst();
      while (!dq.isEmpty() && nums[dq.peekLast()] >= nums[i])
          dq.pollLast();

      dq.offerLast(i);

      if (i >= k - 1)
          res[i - k + 1] = nums[dq.peekFirst()];
    }
    return res;
  }
}
```